Deflection of a simply supported beam under uniform load

Filip Hokeš · 2026-09-04 · version 1.2

Contents

1 Introduction 1
2 Input data and parametric study 2
3 Computational script 4
4 Graphical results 6
5 Source vs. result 7
6 Conclusion 7

Čeština · English · Deutsch

Introduction

This study deals with the deflection of a simply supported beam of span L carrying a uniformly distributed load q. The aim is to compare the closed analytical solution with a numerical solution obtained by the finite-difference method[1] and to show under which conditions the Bernoulli–Navier hypothesis is sufficient. Back in the 1960s such values were read from tables; today a script of a few dozen lines does the job.

The maximum deflection at midspan is given by

$$w_{\max} = \frac{5\,q\,L^4}{384\,E\,I},$$

where E is Young's modulus and I the second moment of area. For steel, $E = 210\,\mathrm{GPa}$; timber is roughly ten times softer. Codes usually limit the relative deflection $w_{\max}/L$ to 1/250.

The formula rests on the Bernoulli–Navier hypothesis, which neglects the shear contribution to deflection. What governs its magnitude is not slenderness as such but the **ratio of bending to shear stiffness**: for a uniform load,

$$\frac{w_{\mathrm{V}}}{w_{\mathrm{M}}} = \frac{q L^2/(8\,G\,A_{\mathrm{s}})}{5\,q\,L^4/(384\,E\,I)} = 9.6\,\frac{E\,I}{G\,A_{\mathrm{s}}\,L^2},$$

where G is the shear modulus and A_{s} the shear area. Only for a homogeneous rectangular section does this reduce to $\approx 2.5\,(h/L)^2$, so that for $h/L = 1/20$ the shear part is about 0.6 % —which is where the popular slenderness rule of thumb comes from. For sandwich, thin-walled or composite sections with a low $G\,A_{\mathrm{s}}$, shear may matter even for a very slender beam and Timoshenko theory is appropriate.

Input data and parametric study

The following table summarises a parametric study of 36 combinations of span and load. The table deliberately runs over one page to exercise the repeated table header in print.

no.	L [m]	q [kN/m]	E [GPa]	I [10⁻⁶ m⁴]	w_max [mm]	w/L [–]
1	3.0	5.0	210	83.3	0.60	1/4970
2	3.0	10.0	210	83.3	1.21	1/2485
3	3.0	15.0	210	83.3	1.81	1/1657
4	3.5	5.0	210	83.3	1.12	1/3133
5	3.5	10.0	210	83.3	2.23	1/1567
6	3.5	15.0	210	83.3	3.35	1/1044
7	4.0	5.0	210	83.3	1.90	1/2100
8	4.0	10.0	210	83.3	3.81	1/1050
9	4.0	15.0	210	83.3	5.71	1/700
10	4.5	5.0	210	83.3	3.05	1/1475
11	4.5	10.0	210	83.3	6.10	1/738
12	4.5	15.0	210	83.3	9.15	1/492
13	5.0	5.0	210	83.3	4.65	1/1075
14	5.0	10.0	210	83.3	9.30	1/538
15	5.0	15.0	210	83.3	13.95	1/358

no.	L [m]	q [kN/m]	E [GPa]	I [10⁻⁶ m⁴]	w_max [mm]	w/L [–]
16	5.5	5.0	210	83.3	6.81	1/808
17	5.5	10.0	210	83.3	13.62	1/404
18	5.5	15.0	210	83.3	20.43	1/269
19	6.0	5.0	210	83.3	9.64	1/622
20	6.0	10.0	210	83.3	19.28	1/311
21	6.0	15.0	210	83.3	28.93	1/207
22	6.5	5.0	210	83.3	13.29	1/489
23	6.5	10.0	210	83.3	26.57	1/245
24	6.5	15.0	210	83.3	39.86	1/163
25	7.0	5.0	210	83.3	17.86	1/392
26	7.0	10.0	210	83.3	35.71	1/196
27	7.0	15.0	210	83.3	53.57	1/131
28	7.5	5.0	210	83.3	23.54	1/319
29	7.5	10.0	210	83.3	47.08	1/159
30	7.5	15.0	210	83.3	70.62	1/106
31	8.0	5.0	210	83.3	30.48	1/262
32	8.0	10.0	210	83.3	60.95	1/131
33	8.0	15.0	210	83.3	91.43	1/87
34	8.5	5.0	210	83.3	38.84	1/219
35	8.5	10.0	210	83.3	77.68	1/109
36	8.5	15.0	210	83.3	116.52	1/73

Results for spans above 8.0 m clearly violate the 1/250 limit and call for a deeper section or a camber.

Computational script

The analytical solution is verified by the following script. The lines are deliberately long to exercise code wrapping in print.

```python
"""Simply supported beam deflection: analytical vs. numerical (finite
  differences)."""

import numpy as np


def analytical_solution(L: float, q: float, E: float, I: float, n:
  int = 101) -> np.ndarray:

    """Return the deflection curve w(x) = q x (L^3 - 2 L x^2 + x^3) /
  (24 E I)."""

    x = np.linspace(0.0, L, n)

    return q * x * (L**3 - 2.0 * L * x**2 + x**3) / (24.0 * E * I)


def numerical_solution(L: float, q: float, E: float, I: float, n: int
  = 101) -> np.ndarray:

    """Solve E I w'''' = q by finite differences with w(0) = w(L) =
  0, w''(0) = w''(L) = 0."""

    h = L / (n - 1)

    A = np.zeros((n, n)); b = np.full(n, q * h**4 / (E * I))

    for i in range(2, n - 2):

        A[i, i-2:i+3] = [1.0, -4.0, 6.0, -4.0, 1.0]

    A[0, 0] = A[n-1, n-1] = 1.0; b[0] = b[n-1] = 0.0
  # w = 0 at supports

    A[1, 0:3] = [1.0, -2.0, 1.0]; b[1] = 0.0
  # w'' = 0 (left hinge)

    A[n-2, n-3:n] = [1.0, -2.0, 1.0]; b[n-2] = 0.0
  # w'' = 0 (right hinge)

    return np.linalg.solve(A, b)
```

```
if __name__ == "__main__":

    L, q, E, I = 6.0, 10_000.0, 210e9, 83.3e-6

    w_a = analytical_solution(L, q, E, I)

    w_n = numerical_solution(L, q, E, I)

    print(f"analytical w_max = {1000 * w_a.max():.3f} mm, numerical
  w_max = {1000 * w_n.max():.3f} mm")

    print(f"relative deviation = {abs(w_n.max() - w_a.max()) /
  w_a.max():.2e}")
```

Graphical results

The deflection curves of both solutions are plotted in the following figure.

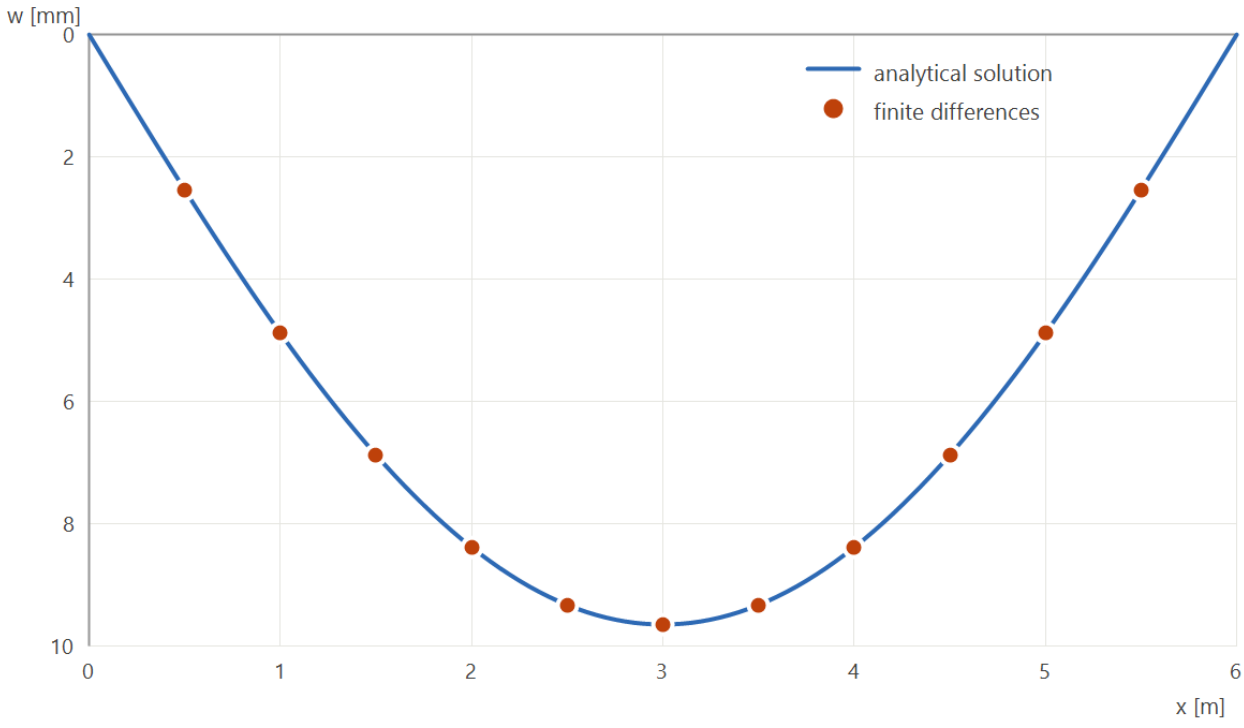


Fig. 1: Deflection curve—analytical vs. numerical solution

The static scheme, stored under the same file name in a different folder:

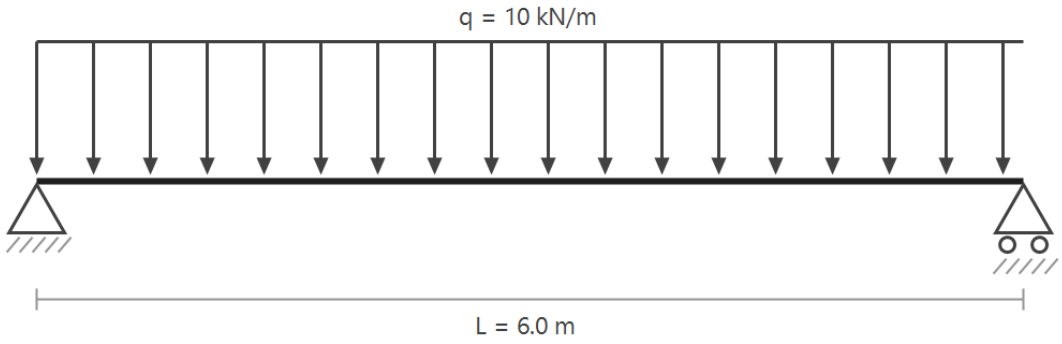


Fig. 2: Static scheme of the simply supported beam

Source vs. result

This document was produced from plain Markdown. The block below is the literal source text (typographic rules are deliberately not applied inside code blocks):

```
She said "the span is 6 m" - and she's right: pages 10-20 of the
  report

list a 40x60 mm section tested at 25 °C... Don't forget section 3.
```

And this is how mdprint typesets the very same text:

She said “the span is 6 m”—and she’s right: pages 10–20 of the report list a 40×60 mm section tested at 25 °C… Don’t forget section 3.

The difference lives in the details that decide legibility: curly quotes, an em dash, an en dash in the range, a real multiplication sign, a proper apostrophe and a bound number–unit pair. Math is typeset; code and paths stay untouched.

Conclusion

The numerical solution differs from the analytical one by less than 0.01 %, confirming both approaches. The inline relation $w(x) = \frac{qx(L^3 - 2Lx^2 + x^3)}{24EI}$ can therefore be used for quick checks[2]. A definition to close with:

Serviceability limit state

A state beyond which the structure no longer meets its operational requirements, although no failure occurs.

---

1. Finite-difference method—the derivatives in the governing differential equation are replaced by difference quotients on a regular grid of nodes. Not to be confused with the finite element method, which seeks an approximate solution in a weak formulation over element basis functions. ↩
2. A design check per EN 1990 requires load combinations, which this text omits for brevity. ↩